Entrega parte 6 grupo 1

Manuel Jusdado Ciber B

Josías González Ciber B

Índice

Explicación pipeline..... 4
Ejecución..... 5
Resultado de la ejecución:..... 5
PIPELINE..... 6

Explicación pipeline

Podemos encontrar el código del pipeline en el siguiente enlace:

https://github.com/grupo1BPEPS/peps-team/blob/master/pipeline

Está compuesto de las siguientes pruebas:

- Pruebas unitarias con pytest sobre tests/unit, generando cobertura con pytest-cov y resultados en pytest_results.xml
- Análisis estático (paralelo):
  - Secretos con detect-secrets → escanea todos los archivos y guarda el resultado en reports/detect-secrets.json
  - Pruebas SAST con SonarQube Scanner → analiza el proyecto con clave peps-team apuntando a Python 3.11
  - Seguridad en IaC con Checkov → analiza Dockerfiles y manifiestos Kubernetes, genera reporte en JUnit XML con –soft-fail
  - SCA (Análisis de composición de software) con OWASP Dependency Check → escanea dependencias, falla si hay CVEs con CVSS ≥ 7, usa clave de API de NVD

- Construye imagen Docker de la app Python
- Escaneo de seguridad del contenedor con Trivy → analiza la imagen Docker recién construida
- Arranque de contenedores → levanta peps-api (Flask, puerto 5000) y peps-web (Apache, puerto 8080) en una red Docker llamada peps-net
- Pruebas E2E con pytest + Selenium sobre tests/e2e, usando Chrome en contenedor sobre los contenedores
- DAST con Nikto → escanea http://localhost:5000 (python app), guarda resultado en JSON en reports/
- Push a DockerHub de la imagen Python (condicional)
- Limpieza → logout de Docker, elimina contenedores y hace docker system prune para eliminar redes y volumenes
- Notificaciones por email → envía correo a josias.gonzalez1@educa.madrid.org tanto en éxito como en fallo del pipeline (en revisión)

Ejecución

- Antes de ejecutar el pipeline hay que levantar el contenedor de sonnarqube

```
ciber2@pcCiber2-25:~/Escritorio/asignaturas/peps/peps-team$ sudo docker start 78044f99a820
78044f99a820
ciber2@pcCiber2-25:~/Escritorio/asignaturas/peps/peps-team$ sudo docker ps -a | grep "sona"
78044f99a820   sonarqube:lts             "/opt/sonarqube/dock…"   2 weeks ago   Up 6 seconds              0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
                                                                                                         sonarqube
```

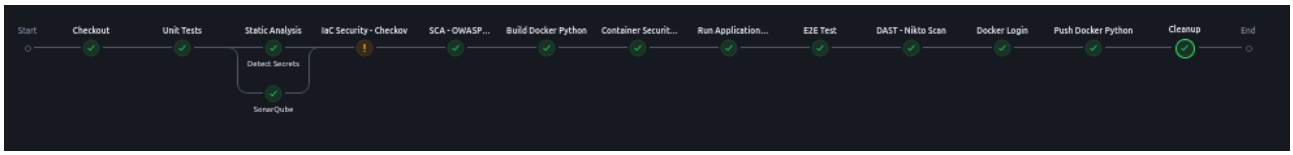
- En algunas ocasiones la ejecución falla porque al descargar el repositorio de git nos bloquea, demasiadas peticiones.

```
        ... 12 more
ERROR: Error fetching remote repo 'origin'
ERROR: Maximum checkout retry attempts reached, aborting
[Pipeline] }
```

- Cada X tiempo hay que solicitar de nuevo la API para las pruebas de owasp-dependency-check, ya que caduca.

```
/usr/share/dependency-check/bin/dependency-check.sh --project "peps-team" --scan . --format JSON --format HTML --out reports --failOnCVSS 7 --nvdApiKey $NVD_KEY
+ /usr/share/dependency-check/bin/dependency-check.sh --project peps-team --scan . --format JSON --format HTML --out reports --failOnCVSS 7 --nvdApiKey ****
[INFO] Checking for updates
[ERROR] Error updating the NVD Data
org.owasp.dependencycheck.data.update.exception.UpdateException: Error updating the NVD Data
        at org.owasp.dependencycheck.data.update.NvdApiDataSource.processApi(NvdApiDataSource.java:387)
        at org.owasp.dependencycheck.data.update.NvdApiDataSource.update(NvdApiDataSource.java:128)
        at org.owasp.dependencycheck.Engine.doUpdates(Engine.java:902)
        at org.owasp.dependencycheck.Engine.initializeAndUpdateDatabase(Engine.java:707)
        at org.owasp.dependencycheck.Engine.analyzeDependencies(Engine.java:634)
        at org.owasp.dependencycheck.App.runScan(App.java:270)
        at org.owasp.dependencycheck.App.run(App.java:202)
        at org.owasp.dependencycheck.App.main(App.java:93)
Caused by: io.github.jeremylong.openvulnerability.client.nvd.NvdApiException: Invalid API Key: 446c1-*****-9dcff
```

Resultado de la ejecución:



PIPELINE

```
pipeline {
    agent any

    parameters {
        string(name: 'IMAGE_TAG_PYTHON', defaultValue: '', description: 'Tag imagen Docker python')
        string(name: 'IMAGE_TAG_WAF', defaultValue: '', description: 'Tag imagen Docker waf')
    }

    environment {
        DOCKER_IMAGE_PYTHON = "sirjousis/peps"
        DOCKER_IMAGE_WAF    = "fernipeps/pepsapachewaf"
        FINAL_IMAGE_TAG_PYTHON = "${params.IMAGE_TAG_PYTHON}"
        FINAL_IMAGE_TAG_WAF    = "${params.IMAGE_TAG_WAF}"
        PYTHON_IMAGE = "python:3.11-slim"
    }

    stages {

        stage('Checkout') {
            steps {
                git url: 'https://github.com/grupo1BPEPS/peps-team.git',
                    branch: 'master',
                    credentialsId: 'GitHub'
            }
        }

        stage('Unit Tests') {

            agent {
                docker {
```

```
            image "${PYTHON_IMAGE}"
            args '-u root'
            reuseNode true
        }
    }

    steps {
        sh '''
        apt-get update
        apt-get install -y chromium

        python -m venv venv
        . venv/bin/activate

        pip install pytest pytest-cov selenium webdriver-manager

        export PYTHONPATH=$WORKSPACE

        pytest tests/unit \
            --cov=api/web \
            --cov-fail-under=1 \
            --junitxml=pytest_results.xml
        '''
    }

    post {
        always {
            junit 'pytest_results.xml'
        }
    }
}
```

```
stage('Static Analysis') {

    parallel {

        stage('Detect Secrets') {

            agent {
                docker {
                    image "${PYTHON_IMAGE}"
                    reuseNode true
                }
            }

            steps {
                sh '''
                pip install detect-secrets
                mkdir -p reports

                detect-secrets scan . --all-files \
                > reports/detect-secrets.json
                '''
            }

            post {
                always {
                    archiveArtifacts artifacts: 'reports/detect-secrets.json'
                }
            }
        }

        stage('SonarQube') {
```

```
            agent {
                docker {
                    image 'sonarsource/sonar-scanner-cli'
                    reuseNode true
                }
            }

            steps {
                withSonarQubeEnv('sonarqube') {
                    sh '''
                    sonar-scanner \
                      -Dsonar.projectKey=peps-team \
                      -Dsonar.sources=. \
                      -Dsonar.python.version=3.11
                    '''
                }
            }
        }
    }
}

stage('IaC Security - Checkov') {

    agent {
        docker {
            image 'bridgecrew/checkov'
            args '--entrypoint="""'
            reuseNode true
        }
    }

    steps {
```

```
            sh '''
            mkdir -p reports

            checkov \
              -d . \
              --framework dockerfile,kubernetes \
              -o junitxml \
              --output-file-path reports \
              --soft-fail
            '''
        }

        post {
            always {
                junit allowEmptyResults: true, testResults: 'reports/results_junitxml.xml'
                archiveArtifacts artifacts: 'reports/results_junitxml.xml'
            }
        }
    }

    stage('SCA - OWASP Dependency Check') {

        environment {
            NVD_KEY = credentials('NVD_API_KEY')
        }

        agent {
            docker {
                image 'owasp/dependency-check'
                args '--entrypoint="" -u root -v
/var/jenkins_home/dependency-check:/usr/share/dependency-check/data'
                reuseNode true
```

```
        }
    }

    steps {
        sh '''
        /usr/share/dependency-check/bin/dependency-check.sh \
            --project "peps-team" \
            --scan . \
            --format JSON \
            --format HTML \
            --out reports \
            --failOnCVSS 7 \
            --nvdApiKey $NVD_KEY
        '''
    }

    post {
        always {
            archiveArtifacts artifacts: 'reports/*'
        }
    }
}

stage('Build Docker Python') {

    when { expression { params.IMAGE_TAG_PYTHON?.trim() } }

    steps {
        script {
            dockerImagePython = docker.build(
                "${DOCKER_IMAGE_PYTHON}:${FINAL_IMAGE_TAG_PYTHON}",
                "-f Dockerfile ."
```

```
            )
        }
    }
}

stage('Container Security Scan (Trivy)') {

    when { expression { params.IMAGE_TAG_PYTHON?.trim() } }

    steps {
        sh '''
        docker run --rm \
        aquasec/trivy image \
        $DOCKER_IMAGE_PYTHON:$FINAL_IMAGE_TAG_PYTHON
        '''
    }
}

stage('Run Application Container') {

when { expression { params.IMAGE_TAG_PYTHON?.trim() } }

steps {
    sh '''
    docker rm -f peps-api || true
    docker rm -f peps-web || true
    docker network rm peps-net || true

    docker network create peps-net

    docker run -d \
        --network peps-net \
```

```
        -p 5000:5000 \
        --name peps-api \
        $DOCKER_IMAGE_PYTHON:$FINAL_IMAGE_TAG_PYTHON

    docker pull sirjousis/pepsapache:1.0

    docker run -d \
        --network peps-net \
        -p 8080:80 \
        --name peps-web \
        sirjousis/pepsapache:1.0

    echo "Esperando a que los servicios arranquen..."
    sleep 10

    echo "Contenedores activos:"
    docker ps

    echo "Logs API:"
    docker logs peps-api || true

    echo "Logs WEB:"
    docker logs peps-web || true
    '''
}
    }


    stage('E2E Test') {
when { expression { params.IMAGE_TAG_PYTHON?.trim() } }
agent {
    docker {
```

```
            image 'selenium/standalone-chrome:latest'
            args '--network peps-net'
            reuseNode true
        }
    }

    steps {
        sh '''
            pip install pytest selenium
            pytest tests/e2e --junitxml=e2e_results.xml
        '''
    }

    post {

        always {

            junit 'e2e_results.xml'

        }

    }

}

        stage('DAST - Nikto Scan') {

            steps {
                sh '''
                mkdir -p reports
                chmod 777 reports
                touch ./reports/nikto-report.json
```

```
        chmod 777 ./reports/nikto-report.json
        docker run --rm \
            -v $(pwd)/reports:/reports \
            -u root \
            sullo/nikto \
            -h http://localhost:5000 \
            -Display V \
            -Format json \
            -output /reports/nikto-report.json
        '''
    }

    post {
        always {
            archiveArtifacts artifacts: 'reports/*'
        }
    }
}

stage('Docker Login') {

    steps {
        withCredentials([usernamePassword(
            credentialsId: 'DockerHub',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {

            sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
        }
    }
}
```

```
        stage('Push Docker Python') {

            when { expression { params.IMAGE_TAG_PYTHON?.trim() } }

            steps {
                script {
                    dockerImagePython.push()
                }
            }
        }

        stage('Cleanup') {

            steps {
                sh '''
                docker logout || true
                docker rm -f peps-api || true
                docker system prune -f || true
                '''
            }
        }
    }

    post {

        success {
            emailext(
                to: 'josias.gonzalez1@educa.madrid.org',
                subject: "SUCCESS: ${currentBuild.fullDisplayName}",
                body: """
```

Pipeline ejecutado correctamente

```
Job: ${env.JOB_NAME}
Build: ${env.BUILD_NUMBER}

${env.BUILD_URL}
"""
            )
        }

        failure {
            emailext(
                to: 'josias.gonzalez1@educa.madrid.org',
                subject: "FAILED: ${currentBuild.fullDisplayName}",
                body: """
Pipeline fallido

Job: ${env.JOB_NAME}
Build: ${env.BUILD_NUMBER}

Revisar:
${env.BUILD_URL}
"""
            )
        }
    }
}
```